



الجمهورية العربية السورية
وزارة التعليم العالي جامعة دمشق
كلية الهندسة المعلوماتية

محرك المعالجة عالي الاداء لنظام التجارة الالكترونية
(High-Performance E-commerce Backend
Engine)

مقرر: البرمجة التفرعية

اعداد الطلاب:

- مادح هاشم معاد
- مجد محمد محمد
- عمر فادي العيسى

مقدمة :

يهدف هذا المشروع إلى تطوير نظام تجارة إلكترونية يعتمد على بنية Backend باستخدام إطار العمل Laravel ، مع التركيز على تحقيق مجموعة من المتطلبات غير الوظيفية

(Non-Functional Requirements)

المتعلقة بالأداء، الاستقرار، قابلية التوسع، وإدارة الموارد.

تم تنفيذ واختبار الحلول باستخدام مجموعة من التقنيات الحديثة مثل:

- Database Transactions

- Row-Level Locking

- Queue Jobs

- Batch Processing

- Rate Limiting

- Load Balancing Simulation

- Apache JMeter

كما تم إجراء اختبارات ضغط (Stress Testing) ومحاكاة لمستخدمين متزامنين للتحقق من فعالية الحلول وتحسين أداء النظام تحت الضغط.

المتطلب الأول :

التحكم بالتزامن ومنع تضارب البيانات (Concurrency Control)

وصف المشكلة

عند قيام عدة مستخدمين بتنفيذ طلبات شراء متزامنة لنفس المنتج، قد تحدث مشاكل في تناقض البيانات (Data Inconsistency) مثل:

- بيع كمية أكبر من المتوفرة في المخزن
- خصم رصيد المستخدم أكثر من مرة
- حدوث Race Conditions

خصوصاً عند تنفيذ الطلبات بشكل متزامن. (Concurrent Requests)

الاستراتيجية المستخدمة

تم استخدام:

- Database Transactions
- Row-Level Locking باستخدام lockForUpdate()

وذلك لضمان:

- تنفيذ العمليات بشكل ذري (Atomic Operations)
- منع تعديل نفس البيانات من أكثر من عملية بنفس الوقت

آلية التنفيذ

عند تنفيذ عملية إنشاء الطلب:

1. يتم قفل سجل المستخدم (User Row Lock)
2. يتم قفل الطلب الحالي
3. يتم قفل المنتج المطلوب
4. يتم التحقق من الكمية المتوفرة
5. يتم خصم الكمية من المخزن
6. يتم خصم الرصيد من المحفظة
7. يتم تأكيد الطلب داخل Transaction واحدة

التقنيات المستخدمة

- MySQL Transactions
- Laravel DB::transaction()
- lockForUpdate()
- Apache JMeter

الاختبار

تم استخدام Apache JMeter لمحاكاة:

- 10 مستخدمين مختلفين
- يطلبون نفس المنتج بنفس اللحظة
- مع كمية مخزون محدودة

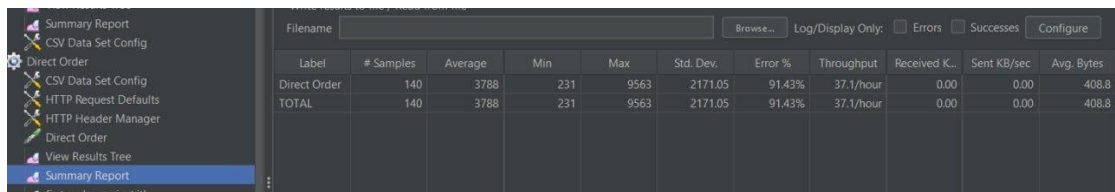
وذلك للتحقق من:

- عدم تجاوز الكمية المتوفرة
- منع تضارب البيانات
- سلامة العمليات المتزامنة

النتائج

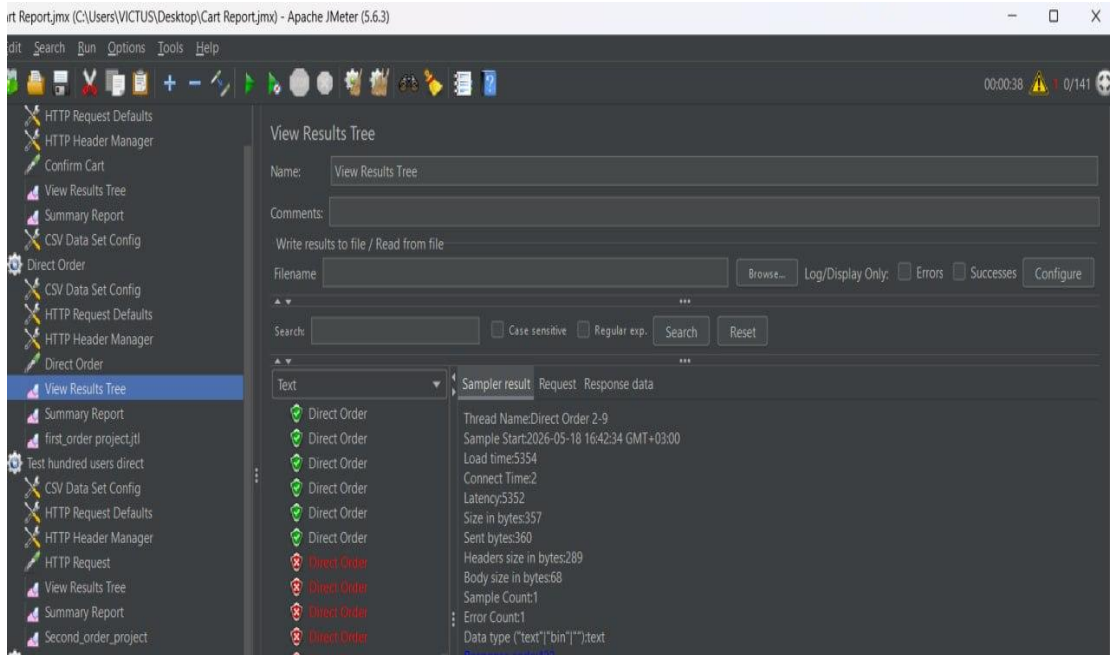
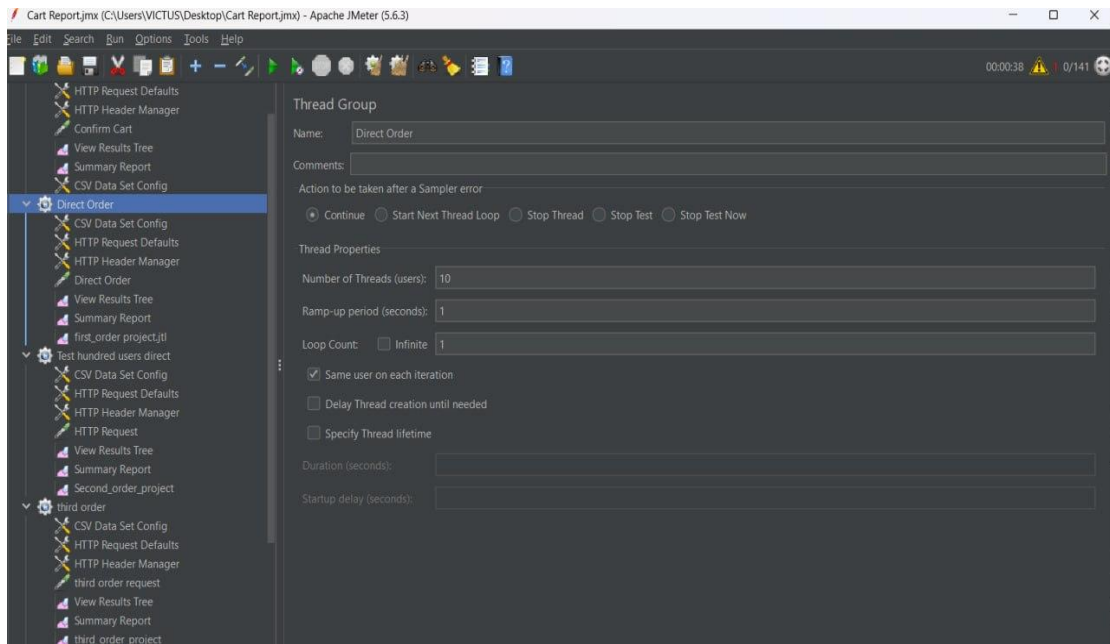
أظهرت النتائج:

- نجاح النظام في منع بيع كمية تتجاوز المخزون
- عدم حدوث أي تضارب بيانات
- تنفيذ العمليات بشكل آمن ومتزامن



The screenshot shows the Apache JMeter Summary Report interface. The left sidebar lists various report options, with 'Summary Report' selected. The main area displays a table of test results for the 'Direct Order' test.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
Direct Order	140	3788	231	9563	2171.05	91.43%	37.1/hour	0.00	0.00	408.8
TOTAL	140	3788	231	9563	2171.05	91.43%	37.1/hour	0.00	0.00	408.8



المتطلب الثاني

إدارة الموارد الحاسوبية والتحكم بعدد العمليات المتوازية

وصف المشكلة

عند استقبال عدد كبير من الطلبات خلال فترة زمنية قصيرة، قد يؤدي ذلك إلى:

- استهلاك مفرط للموارد
- بطء في الاستجابة
- انهيار النظام (System Overload)

الاستراتيجية المستخدمة

تم استخدام:

- Rate Limiting
- Middleware Throttling

للتحكم بعدد الطلبات المسموح بها لكل مستخدم خلال فترة زمنية محددة.

آلية التنفيذ

تم تعريف مجموعة من الـ Rate Limiters مثل:

- cart-write
- wallet-deposit
- admin-write

بحيث يتم:

- تحديد عدد الطلبات المسموح بها بالدقيقة
- منع تجاوز الحد المسموح

التقنيات المستخدمة

- Laravel Rate Limiter
 - Middleware Throttle
 - Apache JMeter
-

الاختبار

تم إجراء اختبارين باستخدام JMeter

قبل تطبيق Throttling

تم إرسال عدد كبير من الطلبات المتزامنة بدون أي قيود.

بعد تطبيق Throttling

تم تفعيل الـ Rate Limiter وإعادة نفس الاختبار.

النتائج

أظهرت النتائج:

- انخفاض استهلاك الموارد
- استقرار أكبر للنظام
- التحكم في عدد العمليات المتزامنة
- منع الضغط المفرط على الخادم

كما ظهر اختلاف واضح في قيمة الـ Throughput بين الحالتين.

	Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
Direct Order	HTTP Reque...	300	9139	777	24684	5985.20	66.67%	46.1/min	0.41	0.27	546.4
	TOTAL	300	9139	777	24684	5985.20	66.67%	46.1/min	0.41	0.27	546.4

نتائج الاختبار قبل الـ Throttling

	Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
Summary Report	HTTP Reque...	400	11453	532	32975	7862.18	50.00%	27.0/min	0.30	0.16	689.9
	TOTAL	400	11453	532	32975	7862.18	50.00%	27.0/min	0.30	0.16	689.9

نتائج الاختبار بعد الـ Throttling

المتطلب الثالث

المعالجة غير المتزامنة (Asynchronous Processing)

وصف المشكلة

بعض العمليات لا يحتاج المستخدم إلى انتظارها مباشرة، مثل:

- إصدار الفواتير
- إنشاء السجلات
- إرسال الإشعارات

تنفيذ هذه العمليات داخل مسار الطلب الرئيسي يؤدي إلى زيادة زمن الاستجابة.

الاستراتيجية المستخدمة

تم استخدام:

- Queue Jobs
- Background Processing

وذلك لنقل العمليات الثانوية إلى الخلفية.

آلية التنفيذ

بعد تأكيد الطلب:

1. يتم إنهاء الطلب وإرجاع الاستجابة للمستخدم مباشرة
2. يتم إرسال Job إلى Queue
3. تقوم الـ Queue بتنفيذ عملية إنشاء الفاتورة بالخلفية

التقنيات المستخدمة

- Laravel Queues
 - ShouldQueue
 - Queue Worker
 - Dispatchable Jobs
-

الاختبار

تم تنفيذ اختبار ضغط باستخدام JMeter لمحاكاة عدة مستخدمين يقومون بتأكيد الطلبات بالتزامن.

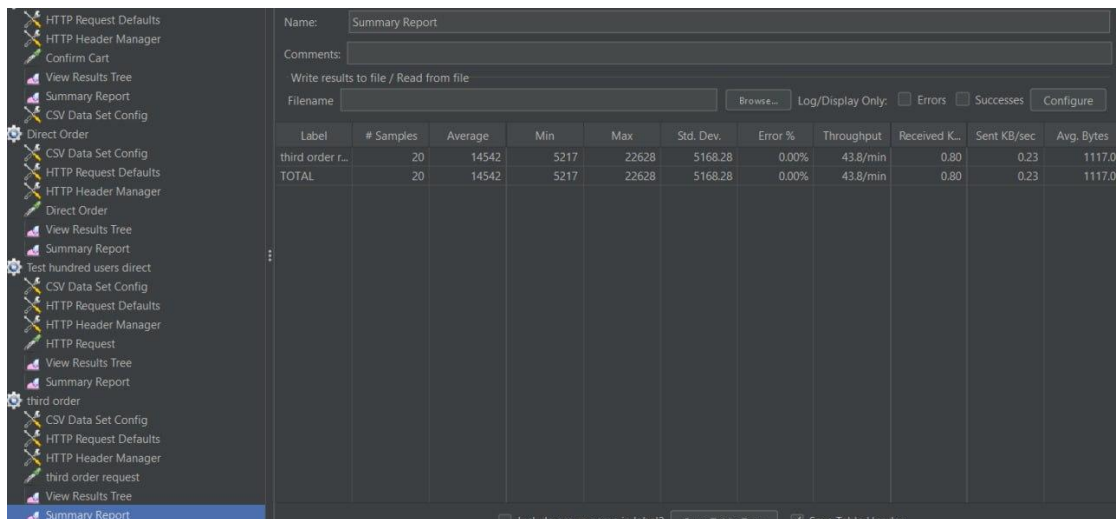
ثم تم مراقبة:

- سرعة استجابة الـ API
- تنفيذ الـ Jobs بالخلفية
- سجلات النظام (Logs)

النتائج

أظهرت النتائج:

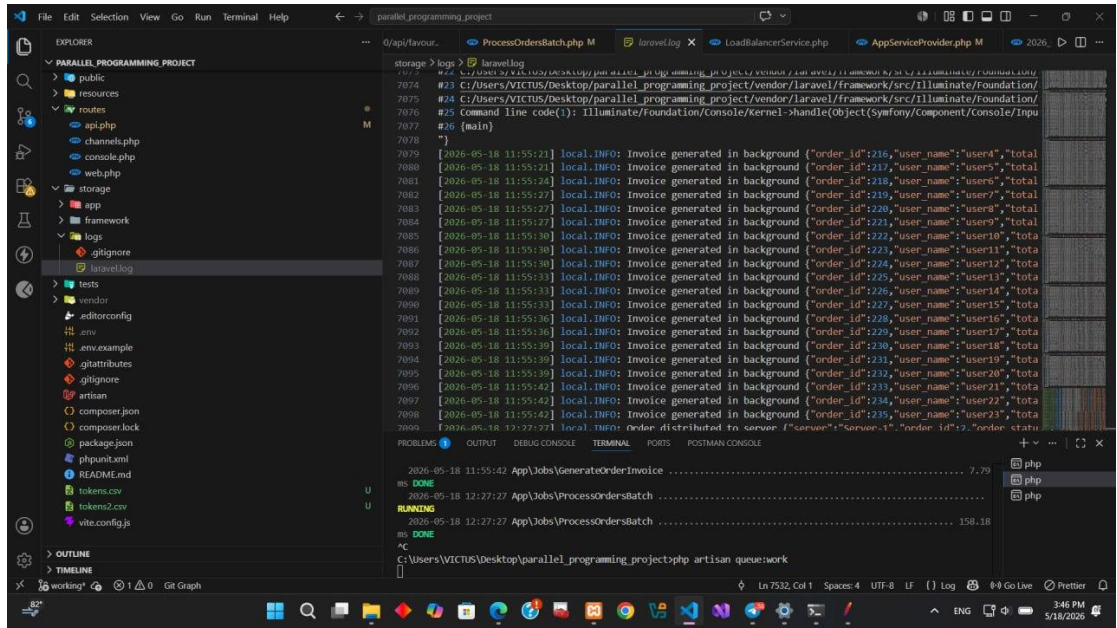
- تقليل زمن استجابة الطلبات
- نقل المهام الثقيلة إلى الخلفية
- تحسين تجربة المستخدم
- استمرار تنفيذ العمليات الثانوية دون التأثير على المستخدم



The screenshot shows the JMeter Summary Report interface. The left sidebar lists various test components, and the main area displays a table of results. The table has columns for Label, # Samples, Average, Min, Max, Std. Dev., Error %, Throughput, Received K..., Sent KB/sec, and Avg. Bytes. The data is organized into a tree structure, with 'third order r...' and 'TOTAL' being the visible nodes.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
third order r...	20	14542	5217	22628	5168.28	0.00%	43.8/min	0.80	0.23	1117.0
TOTAL	20	14542	5217	22628	5168.28	0.00%	43.8/min	0.80	0.23	1117.0

نتائج JMeter



Logs الخاصة بإنشاء الطلب بالخلفية

المتطلب الرابع

معالجة البيانات الضخمة (Batch Processing)

وصف المشكلة

معالجة عدد كبير من الطلبات دفعة واحدة قد يؤدي إلى:

- استهلاك مرتفع للذاكرة
- انخفاض الأداء
- بطء التنفيذ

الاستراتيجية المستخدمة

تم استخدام:

- Batch Processing
- Chunk Processing

عبر الدالة:

`chunkById()`

آلية التنفيذ

تم إنشاء Job تقوم بـ:

1. قراءة الطلبات المكتملة
 2. تقسيمها إلى دفعات (Chunks)
 3. معالجة كل دفعة بشكل مستقل
- وذلك لتقليل استهلاك الذاكرة وتحسين الأداء.

التقنيات المستخدمة

- Laravel Queue Jobs
- chunkById()
- Background Processing

الاختبار

تم:

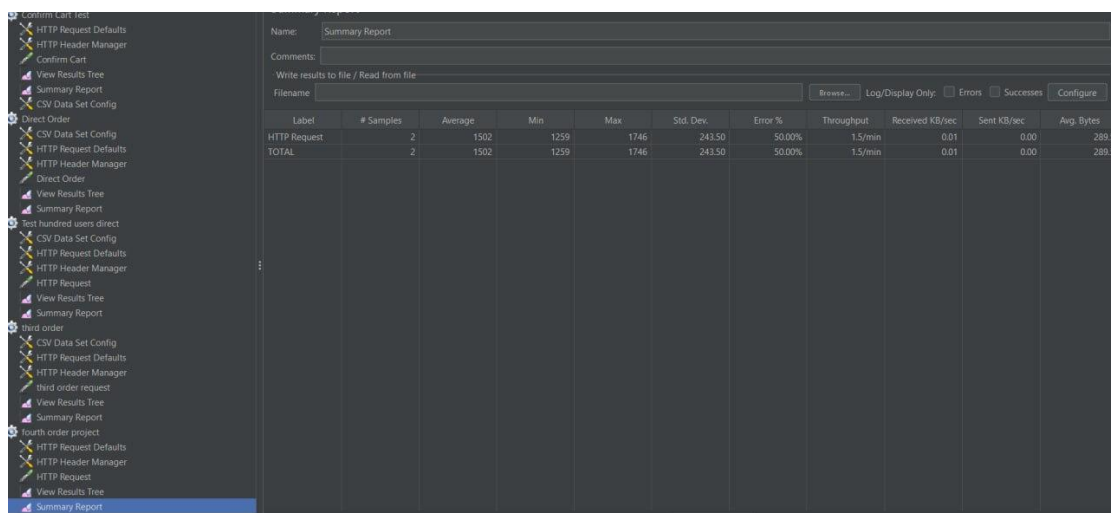
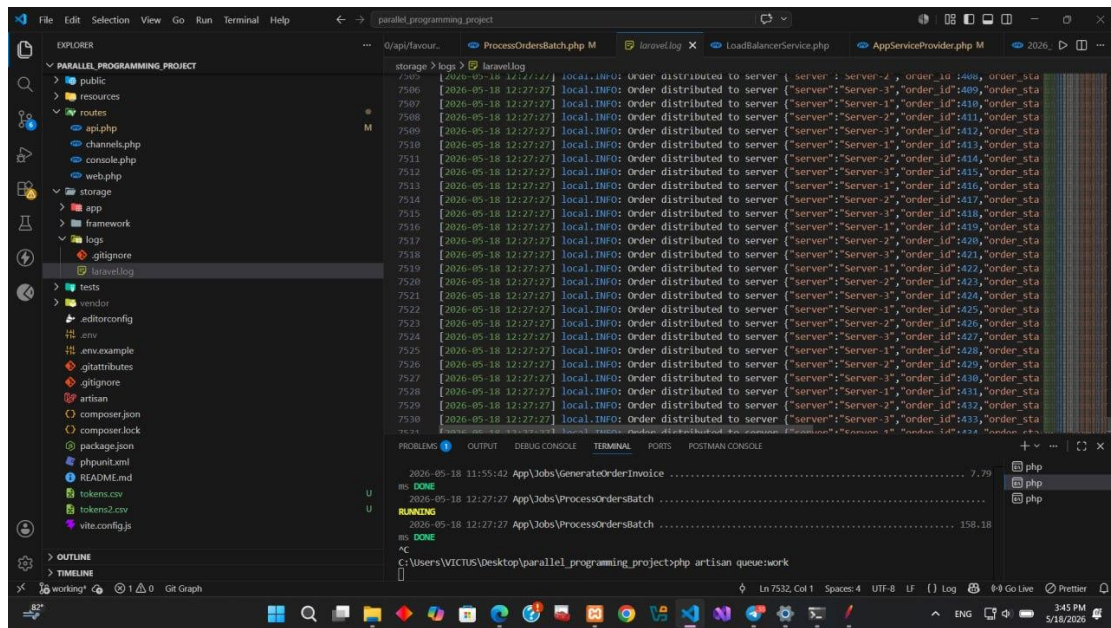
- إنشاء عدد كبير من الطلبات المكتملة
- تشغيل الـ Batch Job
- مراقبة الـ Logs الناتجة عن معالجة كل دفعة

النتائج

أظهرت النتائج:

- معالجة الطلبات على دفعات صغيرة
- تقليل استهلاك الذاكرة
- تحسين قابلية التوسع (Scalability)

كما تم تسجيل عدد الطلبات داخل كل دفعة في الـ Logs.



المتطلب الخامس

محاكاة توزيع الطلبات على أكثر من خادم (Load Balancing Simulation)

وصف المشكلة

عند وجود عدد كبير من الطلبات، فإن إرسال جميع العمليات إلى خادم واحد يؤدي إلى:

- زيادة الضغط على الخادم
- انخفاض الأداء
- ضعف قابلية التوسع

الاستراتيجية المستخدمة

تم استخدام:

- Load Balancing Simulation
- استراتيجية Round Robin

لتوزيع الطلبات بالتناوب على عدة خوادم افتراضية.

آلية التنفيذ

تم إنشاء:

LoadBalancerService

يقوم بتوزيع الطلبات بالتسلسل على:

- Server-1
- Server-2
- Server-3

بحيث يتم إرسال كل طلب إلى خادم مختلف بشكل دوري.

سبب اختيار Round Robin

تم اختيار استراتيجية Round Robin لأنها:

- بسيطة وسريعة
- تحقق توزيعًا متوازنًا
- مناسبة لمحاكاة أنظمة التوزيع
- لا تحتاج حسابات معقدة

التقنيات المستخدمة

- Service Layer
- Queue Jobs
- Round Robin Algorithm

الاختبار

تم دمج محاكاة التوزيع داخل عملية الـ Batch Processing.

وأظهرت الـ Logs:

- انتقال الطلبات بالتناوب بين الخوادم الثلاثة

النتائج

أظهرت النتائج:

- نجاح توزيع الطلبات على عدة خوادم
- تحقيق توازن في توزيع العمليات
- محاكاة بيئة موزعة (Distributed Environment)

```
storage > logs > laravelLog
11041 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-3","order_id":709,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11042 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-1","order_id":710,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11043 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-2","order_id":711,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11044 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-3","order_id":712,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11045 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-1","order_id":713,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11046 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-2","order_id":714,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11047 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-3","order_id":715,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11048 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-1","order_id":716,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11049 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-2","order_id":717,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11050 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-3","order_id":718,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11051 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-1","order_id":719,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11052 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-2","order_id":720,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11053 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-3","order_id":721,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11054 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-1","order_id":722,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11055 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-2","order_id":723,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11056 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-3","order_id":724,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11057 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-1","order_id":725,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11058 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-2","order_id":726,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11059 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-3","order_id":727,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11060 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-1","order_id":728,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11061 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-2","order_id":729,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11062 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-3","order_id":730,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11063 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-1","order_id":731,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11064 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-2","order_id":732,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11065 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-3","order_id":733,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11066 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-1","order_id":734,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11067 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-2","order_id":735,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11068 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-3","order_id":736,"order_status":"completed","is_paid":true,"payment_status":"completed"}
11069 [2026-05-18 13:42:37] local.INFO: Order distributed to server {"server":"Server-1","order_id":737,"order_status":"completed","is_paid":true,"payment_status":"completed"}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
"118|KqkQmKCEe01rUJHwXl0BRMEU9AKQjngEbx32Ih2F3ce38",
"119|y3QqiehZ0PZhl6GmJBYAT4XC9FIW2HkHLSXhd74aaa2d19",
"120|7V6mGZ8743CLLWJDFJ8pZ8RfomGyppjBxrr60ffca976",
]
```

الخاتمة

تم خلال هذا المشروع تطبيق مجموعة متقدمة من المتطلبات غير الوظيفية بهدف تحسين:

- الأداء
- الاستقرار
- التزامن
- إدارة الموارد
- قابلية التوسع

كما تم استخدام أدوات وتقنيات حديثة لاختبار وتحليل أداء النظام تحت الضغط، مما ساهم في بناء نظام أكثر كفاءة وموثوقية.

وقد أثبتت الاختبارات العملية باستخدام Apache JMeter نجاح الحلول المطبقة في تحقيق الأهداف المطلوبة وتحسين أداء النظام في مختلف السيناريوهات.